

revalidator线程删除mask不一致datapath flow分析

现象

bpfttrace定位

通过datapath mask和xlate mask来打印flow

ovs trace

进一步查看是在哪个table设置的nd_target和nd_ttl

ovs相关patch

datapath flow和生成的megaflow的mask不一致，为什么revalidator没有持续快速的删除呢？

现象

两台ipv6的虚拟机：vm1和vm2（假设两个虚拟机位于不同节点上），在vm1上ping vm2的ipv6地址，发现报文有时会出现报文时延长的情况。

```
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18581 ttl=64 time=0.406 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18582 ttl=64 time=0.364 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18583 ttl=64 time=0.437 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18584 ttl=64 time=0.391 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18585 ttl=64 time=0.861 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18586 ttl=64 time=0.370 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18587 ttl=64 time=0.355 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18588 ttl=64 time=0.397 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18589 ttl=64 time=0.427 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18590 ttl=64 time=0.426 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18591 ttl=64 time=0.408 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18592 ttl=64 time=0.390 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18593 ttl=64 time=1.49 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18594 ttl=64 time=0.339 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18595 ttl=64 time=0.627 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18596 ttl=64 time=0.945 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18597 ttl=64 time=0.403 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18598 ttl=64 time=0.423 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18599 ttl=64 time=0.380 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18600 ttl=64 time=0.837 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18601 ttl=64 time=0.398 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18602 ttl=64 time=1.64 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18603 ttl=64 time=0.415 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18604 ttl=64 time=0.441 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18605 ttl=64 time=0.429 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18606 ttl=64 time=0.404 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18607 ttl=64 time=0.520 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18608 ttl=64 time=1.38 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18609 ttl=64 time=0.484 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18610 ttl=64 time=0.508 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18611 ttl=64 time=0.463 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18612 ttl=64 time=0.476 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18613 ttl=64 time=0.382 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18614 ttl=64 time=0.397 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18615 ttl=64 time=0.359 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18616 ttl=64 time=0.370 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18617 ttl=64 time=0.384 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18618 ttl=64 time=0.382 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18619 ttl=64 time=0.403 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18620 ttl=64 time=0.463 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18621 ttl=64 time=0.371 ms
64 bytes from fd1f:9e62:3182:608c::13: icmp_seq=18622 ttl=64 time=0.378 ms
```

相关jira：[🔥 EAS-137711: 【IPv6】同vpc跨子网下IPv6延迟波动](#) 正在进行

通过观察datapath flow流表，发现 `recirc_id(0)` 的dp flow时不时被删除。

```
1 ufid:3eef51b1-079a-4881-bf41-1dfad6cda5b0, recirc_id(0), dp_hash(0/0), skb
2
3 ufid:69eb1b25-1003-4158-a406-c1758f60c701, recirc_id(0x2), dp_hash(0/0), s
```

通过观察上面两条dp flow，可以看出 `recirc_id(0)` 的dp flow的 `packets:126`，`recirc_id(0x2)` 的dp flow的 `packets:1611`。按照原理，如果dp flow不被删除，`recirc_id(0)` 和 `recirc_id(0x2)` 的dp flow的 `packets` 应该是差不多的。我们通过多次查询和观察，我们可以确定上面的 `recirc_id(0)` 时不时会被删除。

bpfttrace定位

- 查找bpf可以trace的points

```
1 [root@node-4 ~]# bpfttrace -lp $(pidof ovs-vswitchd) "*" | grep revalidat
2 usdt:/proc/55347/root/usr/sbin/ovs-vswitchd:revalidate:flow_result
3 usdt:/proc/55347/root/usr/sbin/ovs-vswitchd:revalidate_ukey__:entry
4 usdt:/proc/55347/root/usr/sbin/ovs-vswitchd:revalidate_ukey__:exit
5 usdt:/proc/55347/root/usr/sbin/ovs-vswitchd:revalidator_sweep__:flow_swe
6 usdt:/proc/55347/root/usr/sbin/ovs-vswitchd:udpif_revalidator:start_dump
7 usdt:/proc/55347/root/usr/sbin/ovs-vswitchd:udpif_revalidator:sweep_done
```

- bpfttrace代码

```
1 #!/usr/bin/env bpfttrace
2
3 #include "ovs.h"
4
5 BEGIN
6 {
7     printf("Tracing ovs-vswitchd revalidate:flow_result usdt... Hit Ctrl-C to end.\n");
8 }
9
10 usdt:/proc/55347/root/usr/sbin/ovs-vswitchd:revalidate:flow_result
11 {
12     $udpif = (struct udpif *)arg0;
13     $udpif_key = (struct udpif_key *)arg1;
14     $reval_result = arg2;
15     $flow_del_reason = arg3;
16
17     // UFID 是 16 字节, 按照 u32[4] 读取
18     $ufid0 = $udpif_key->ufid.u32[0];
19     $ufid1 = $udpif_key->ufid.u32[1];
20     $ufid2 = $udpif_key->ufid.u32[2];
21     $ufid3 = $udpif_key->ufid.u32[3];
22
23     $req_ufid = (uint32)0x3eef51b1;
24
25     if ($ufid0 == $req_ufid) {
26         printf("- [%d@%s] ufid: %08x-%04x-%04x-%04x-%08x reval_resu
27             tid, comm,
28             $ufid0, ($ufid1 >> 16) & 0xffff, $ufid1 & 0xffff, ($ufid2 >
29             $reval_result, $flow_del_reason
30         );
31     }
32 }
```

其中 `ovs.h` 文件是ovs中所有的数据结构，在这里用于bpfttrace解析相关数据结构中的数据。



这里的时不时会打印：reval_result为1,flow_del_reason为5。

```
1 [root@node-4 bpf]# bpfttrace fr.bt
2 Attaching 2 probes...
3 Tracing ovs-vswitchd revalidate:flow_result usdt... Hit Ctrl-C to end.
4 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reval
5 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reval
6 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reval
7 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reval
8 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reval
9 ...
10 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reval
11 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reval
12 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reval
13 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reval
```

```

14 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reva
15 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reva
16 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reva
17 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reva
18 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reva
19 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reva
20 - [55366@revalidator18] ufid: 3eef51b1-079a-4881-bf41-1dfad6cda5b0 reva

```

根据下面的代码可以看出，`reval_result`为`UKEY_DELETE`,`flow_del_reason`为`FDR_FLOW_WILDCARDED`

```

1  enum reval_result {
2      UKEY_KEEP,
3      UKEY_DELETE,
4      UKEY_MODIFY
5  };
6
7  /* Ukey delete reasons used by USDT probes. Please keep in sync with the
8   * definition in utilities/usdt-scripts/flow_reval_monitor.py. */
9  enum flow_del_reason {
10     FDR_NONE = 0,          /* No delete reason specified. */
11     FDR_AVOID_CACHING,     /* Cache avoidance flag set. */
12     FDR_BAD_ODP_FIT,       /* Bad ODP flow fit. */
13     FDR_FLOW_IDLE,        /* Flow idle timeout. */
14     FDR_FLOW_LIMIT,       /* Kill all flows condition reached. */
15     FDR_FLOW_WILDCARDED,  /* Flow needs a narrower wildcard mask. */
16     FDR_NO_OFPROTO,       /* Bridge not found. */
17     FDR_PURGE,            /* User requested flow deletion. */
18     FDR_TOO_EXPENSIVE,     /* Too expensive to revalidate. */
19     FDR_UPDATE_FAIL,      /* Datapath update failed. */
20     FDR_XLATION_ERROR,    /* Flow translation error. */
21     FDR_FLOW_MISSING_DP,  /* Flow is missing from the datapath. */
22 };

```

所以可以确认是因为wc不一致导致 `recirc_id(0)` 的dp flow被删除了。

通过查询 `FDR_FLOW_WILDCARDED`，只有在 `revalidate_ukey__` 函数这里才会设置`del_reason`为 `FDR_FLOW_WILDCARDED`。

```

1  /* Do not modify if any bit is wildcarded by the installed datapath
2   * but not the newly revalidated wildcard mask (wc), i.e., if revali
3   * tells that the datapath flow is now too generic and must be narro
4   * down. Note that we do not know if the datapath has ignored any o
5   * wildcarded bits, so we may be overly conservative here. */
6  if (flow_wildcards_has_extra(&dp_mask, ctx.wc)) {
7      *del_reason = FDR_FLOW_WILDCARDED;
8      goto exit;
9  }

```

有些底层的问题定位，可以采用 `bpftrace` 来进行定位。比如之前定位的问题：

- ✖ [EAS-137708: 【公网IP】41环境独立计算节点创建云主机绑定公网IP网络不通](#) 正在进行 这个可以定位到在 `__udp4_lib_rcv` 函数因为udp隧道报文的checksum不正确而drop报文。
- 🔴 [ECS-7346: CLONE - 层峰网络Zenlayer-墨尔本-MEL1\(CSE-3779\)-9000以上数据包一会通一会不通](#) 已关闭 分析同一个报文在br-int和br-vxlan处理时，是否使用了同一个CPU。

通过datapath mask和xlate mask来打印flow

在 `revalidate_ukey__` 函数中进行通过datapath mask和xlate mask来打印flow。

```

1      struct flow flow;
2      struct flow_wildcards wcx;
3      struct match match;

```

```

4     odp_flow_key_to_flow(ukey->key, ukey->key_len, &flow, NULL);
5     odp_flow_key_to_mask(ukey->mask, ukey->mask_len, &wcx, &flow, N
6     match_init(&match, &flow, &wcx);
7     char *s1 = match_to_string(&match, NULL, OFP_DEFAULT_PRIORITY);
8     VLOG_INFO("----- flow using datapath mask: %s", s1);
9
10    struct match match2;
11    match_init(&match2, &flow, ctx.wc);
12    char *s2 = match_to_string(&match2, NULL, OFP_DEFAULT_PRIORITY)
13    VLOG_INFO("----- flow using xlate mask: %s", s2);
14    free(s2);
15    free(s1);

```

```

1
2 2025-12-18T06:59:47.008Z|02114|ofproto_dpif_upcall(revalidator18)|INFO|-
3
4
5 2025-12-18T06:59:47.008Z|02115|ofproto_dpif_upcall(revalidator18)|INFO|-
6

```

可以看出在打印flow时，datapath mask比xlate mask少了 `nd_target` 和 `nd_ttl`。而且按照原理，xlate mask也不应该设置 `nd_target` 和 `nd_ttl`。因为我们的报文是icmpv6，即icmp_type=128,icmp_code=0。

ovs trace

```

1 [root@node-4 bpf]# ovs-appctl ofproto/trace 'recirc_id(0),dp_hash(0/0)
2 Flow: icmp6,in_port=8,vlan_tci=0x0000,dl_src=fa:16:3e:ce:bd:06,dl_dst=
3
4 bridge("br-int")
5 -----
6 0. in_port=8, priority 100, cookie 0xc51dc48d
7   set_field:0xd/0xffff->reg13
8   set_field:0xb->reg11
9   set_field:0xa->reg12
10  set_field:0x4->metadata
11  set_field:0x5->reg14
12  set_field:0/0xffff0000->reg13
13  resubmit(,8)
14 8. metadata=0x4, priority 50, cookie 0xa3a3a633
15   set_field:0/0x1000->reg10
16   resubmit(,73)
17 73. ipv6,reg14=0x5,metadata=0x4,dl_src=fa:16:3e:ce:bd:06,ipv6_src=
18     resubmit(,74)
19 74. No match.
20     drop
21     move:NXM_NX_REG10[12]->NXM_NX_XXREG0[111]
22     -> NXM_NX_XXREG0[111] is now 0
23     resubmit(,9)
24 9. metadata=0x4, priority 0, cookie 0x10b6d331
25     resubmit(,10)
26 10. metadata=0x4, priority 0, cookie 0x85b2e9f1
27     resubmit(,11)
28 11. metadata=0x4, priority 0, cookie 0x4dd51610
29     resubmit(,12)
30 12. ipv6,metadata=0x4, priority 100, cookie 0xb7e4af52
31     set_field:0x10000000000000000000000000000000/0x10000000000000000000000000000000->
32     resubmit(,13)
33 13. metadata=0x4, priority 0, cookie 0x4ba8ec35
34     resubmit(,14)
35 14. ipv6,reg0=0x1/0x1,metadata=0x4, priority 100, cookie 0xa10f7aeb
36     ct(table=15,zone=NXM_NX_REG13[0..15])
37     drop
38     -> A clone of the packet is forked to recirculate. The forked pip
39     -> Sets the packet to an untracked state, and clears all the conn
40
41 Final flow: icmp6,reg0=0x1,reg11=0xb,reg12=0xa,reg13=0xd,reg14=0x5,met
42 Megaf1ow: recirc_id=0,eth,icmp6,in_port=8,d1_src=fa:16:3e:ce:bd:06,d1_
43 Datapath actions: ct(zone=13),recirc(0x2)
44

```

```

45 =====
46 recirc(0x2) - resume conntrack with default ct_state=trk|new (use --ct
47 =====
48
49 Flow: recirc_id=0x2,ct_state=new|trk,ct_zone=13,eth,icmp6,reg0=0x1,reg
50
51 bridge("br-int")
52 -----
53     thaw
54         Resuming from table 15
55 15. ct_state=new-est+trk,metadata=0x4, priority 7, cookie 0x50bada42
56     set_field:0x80000000000000000000000000000000/0x80000000000000000000000000000000
57     set_field:0x20000000000000000000000000000000/0x20000000000000000000000000000000
58     resubmit(,16)
59 16. ipv6,reg0=0x80/0x80,reg14=0x5,metadata=0x4, priority 2002, cookie
60     set_field:0x10000000000000/0x1000000000000->xreg4
61     set_field:0x20000000000000000000000000000000/0x20000000000000000000000000000000-
62     resubmit(,17)
63 17. metadata=0x4, priority 0, cookie 0x8074e5e4
64     resubmit(,18)
65 18. reg8=0x10000/0x10000,metadata=0x4, priority 1000, cookie 0x34f002a
66     set_field:0/0x1000000000000->xreg4
67     set_field:0/0x2000000000000->xreg4
68     set_field:0/0x4000000000000->xreg4
69     resubmit(,19)
70 19. metadata=0x4, priority 0, cookie 0x72505ace
71     resubmit(,20)
72 20. metadata=0x4, priority 0, cookie 0xd8ae6d5f
73     resubmit(,21)
74 21. metadata=0x4, priority 0, cookie 0x4770e8aa
75     resubmit(,22)
76 22. metadata=0x4, priority 0, cookie 0xfc26e019
77     resubmit(,23)
78 23. metadata=0x4, priority 0, cookie 0x905e790d
79     resubmit(,24)
80 24. metadata=0x4, priority 0, cookie 0x859db510
81     resubmit(,25)
82 25. metadata=0x4, priority 0, cookie 0xf6d37106
83     resubmit(,26)
84 26. metadata=0x4, priority 0, cookie 0xb2e02e55
85     resubmit(,27)
86 27. metadata=0x4, priority 0, cookie 0x66644bef
87     resubmit(,28)
88 28. metadata=0x4, priority 0, cookie 0x3881cbc4
89     set_field:0/0x1000000000000->xreg4
90     set_field:0/0x2000000000000->xreg4
91     set_field:0/0x4000000000000->xreg4
92     resubmit(,29)
93 29. ipv6,reg0=0x2/0x2002,metadata=0x4, priority 100, cookie 0x817da267
94     ct(commit,zone=NXM_NX_REG13[0..15],nat(src),exec(set_field:0/0x1->
95     nat(src)
96     set_field:0/0x1->ct_mark
97     -> Sets the packet to an untracked state, and clears all the conn
98     resubmit(,30)
99 30. metadata=0x4, priority 0, cookie 0xaf1fc7da
100     resubmit(,31)
101 31. metadata=0x4, priority 0, cookie 0xf38f16a6
102     resubmit(,32)
103 32. metadata=0x4, priority 0, cookie 0xb200708
104     resubmit(,33)
105 33. metadata=0x4, priority 0, cookie 0xbc3c94b2
106     resubmit(,34)
107 34. metadata=0x4, priority 0, cookie 0x40e12b53
108     resubmit(,35)
109 35. metadata=0x4, priority 0, cookie 0x92afc5d7
110     resubmit(,36)
111 36. metadata=0x4,dl_dst=fa:16:3e:88:32:5c, priority 50, cookie 0x2e00e
112     set_field:0x7->reg15
113     resubmit(,40)
114 40. priority 0
115     resubmit(,42)

```

```

116 42. reg13=0/0xffff0000,reg15=0x7,metadata=0x4, priority 100, cookie 0x
117   set_field:0x4/0xffffffff->tun_id
118   set_field:0x7->tun_metadata0
119   move:NXM_NX_REG14[0..14]->NXM_NX_TUN_METADATA0[16..30]
120   -> NXM_NX_TUN_METADATA0[16..30] is now 0x5
121   output:1
122   -> output to kernel tunnel
123   resubmit(,43)
124 43. priority 0
125   drop
126
127 Final flow: recirc_id=0x2,eth,icmp6,reg0=0x283,reg11=0xb,reg12=0xa,reg
128 Megaflow: recirc_id=0x2,ct_state=new-est-rel-rpl+trk,ct_mark=0/0x1,et
129 Datapath actions: ct(commit,zone=13,mark=0/0x1,nat(src)),set(tunnel(tu
130 [root@node-4 bpf]#
131

```

通过查看 `recirc 0` 生成的megaflow，的确有`nd_target`和`nd_ttl`字段：

```

1 Megaflow: recirc_id=0,eth,icmp6,in_port=8,d1_src=fa:16:3e:ce:bd:06,d1_ds

```

但是我们最终生成的dp flow如下：

```

1 ufid:3eef51b1-079a-4881-bf41-1dfad6cda5b0, recirc_id(0),dp_hash(0/0),skb

```

生成的dp flow是没有`nd_target`和`nd_ttl`字段的，因为在下发到内核之前，会校验这个flow是否为以下两个类型的icmp6：

```

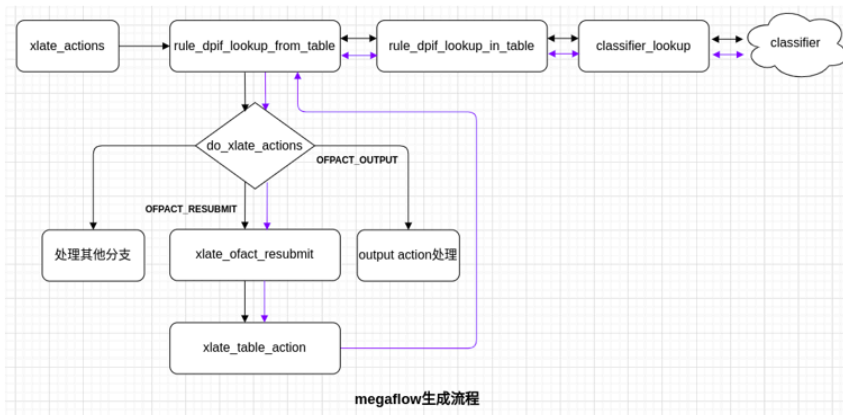
1 #define ND_NEIGHBOR_SOLICIT      135    /* neighbor solicitation */
2 #define ND_NEIGHBOR_ADVERT      136    /* neighbor advertisement */

```

只有这两个类型的icmp6，才会设置`nd_target`和`nd_ttl`字段。因为我们的icmp6类型是128，所以最终下发的dp flow是没有`nd_target`和`nd_ttl`字段的。

而revalidator线程会通过对比xlate的mask和dp flow的mask，发现xlate的mask比dp flow的mask多了`nd_target`和`nd_ttl`字段，所以最终会将dp flow删除了。

进一步查看是在哪个table设置的 `nd_target` 和 `nd_ttl`



在每个table中查找到match的rule后，都会设置 `flow_wildcards` 数据结构的 `wc` 变量，而且 `wc` 变量内容在通过每个table后进行累积，直到最后一个table处理完成，输出的megaflow流表。

我们在 `xlate_actions` 函数的 `rule_dpif_lookup_from_table` 函数内部，打印从classifier中通过生成的wc来打印flow，查看在哪个table设置的 `nd_target` 和 `nd_ttl`。

```

1     if (rule) {
2         struct match match;
3         match_init(&match, flow, wc);
4         char *s1 = match_to_string(&match, NULL, OFP_DEFAULT_PRIORI

```



```

5      VLOG_INFO("***** find table: %d rule_dpif_lookup_i
6      free(s1);
7
8      if (flow->in_port.ofp_port == 17) {
9          struct match match;
10         match_init(&match, flow, wc);
11         char *s1 = match_to_string(&match, NULL, OFP_DEFAULT_PR
12         VLOG_INFO("***** find table: %d rule_dpif_look
13         free(s1);
14
15     }
16     goto out; /* Match. */
17 }

```

```

1 2025-12-18T06:59:07.337Z|00076|ofproto_dpif_xlate(revalidator18)|INFO|-
2 2025-12-18T06:59:07.337Z|00077|ofproto_dpif(revalidator18)|INFO|*****
3 2025-12-18T06:59:07.337Z|00078|ofproto_dpif(revalidator18)|INFO|*****
4 2025-12-18T06:59:07.337Z|00079|ofproto_dpif(revalidator18)|INFO|*****
5 2025-12-18T06:59:07.337Z|00080|ofproto_dpif(revalidator18)|INFO|*****
6 2025-12-18T06:59:07.337Z|00081|ofproto_dpif(revalidator18)|INFO|*****
7 2025-12-18T06:59:07.337Z|00082|ofproto_dpif(revalidator18)|INFO|*****
8 2025-12-18T06:59:07.337Z|00083|ofproto_dpif(revalidator18)|INFO|*****
9 2025-12-18T06:59:07.337Z|00084|ofproto_dpif(revalidator18)|INFO|*****
10 2025-12-18T06:59:07.337Z|00085|ofproto_dpif(revalidator18)|INFO|*****
11 2025-12-18T06:59:07.337Z|00086|ofproto_dpif(revalidator18)|INFO|*****

```

这里可以看出是在table 74设置的。我们查看对应的openflow流表

```

1 [root@node-4 bpf]# ovs-appctl ofproto/detrace ufid:3eef51b1-079a-4881-b
2 cookie=0xc51dc48d, duration=1331s, n_packets=1471, n_bytes=168458, n_of
3 cookie=0xa3a3a633, table_id=8, duration=1331s, n_packets=1483, n_bytes=
4 cookie=0xc51dc48d, table_id=73, duration=1331s, n_packets=1364, n_bytes
5 table_id=254, duration=1331s, n_packets=14374, n_bytes=911972, n_offloa
6 cookie=0x10b6d331, table_id=9, duration=1331s, n_packets=1475, n_bytes=
7 cookie=0x85b2e9f1, table_id=10, duration=1331s, n_packets=1475, n_bytes
8 cookie=0x4dd51610, table_id=11, duration=1331s, n_packets=1475, n_bytes
9 cookie=0xb7e4af52, table_id=12, duration=1331s, n_packets=1328, n_bytes
10 cookie=0x4ba8ec35, table_id=13, duration=1331s, n_packets=1328, n_bytes
11 cookie=0xa10f7aeb, table_id=14, duration=1331s, n_packets=1328, n_bytes
12 [root@node-4 bpf]#
13 [root@node-4 bpf]# ovs-ofctl dump-flows br-int table=73
14 cookie=0xc51dc48d, duration=1346.271s, table=73, n_packets=0, n_bytes=
15 cookie=0x956323ac, duration=1346.266s, table=73, n_packets=0, n_bytes=
16 cookie=0xc51dc48d, duration=1346.266s, table=73, n_packets=0, n_bytes=
17 cookie=0xc51dc48d, duration=1346.265s, table=73, n_packets=0, n_bytes=
18 cookie=0xc51dc48d, duration=1346.259s, table=73, n_packets=0, n_bytes=
19 cookie=0xc51dc48d, duration=1346.266s, table=73, n_packets=1379, n_byt
20 cookie=0xc51dc48d, duration=1346.261s, table=73, n_packets=108, n_byte
21 cookie=0x956323ac, duration=1346.264s, table=73, n_packets=0, n_bytes=
22 cookie=0x956323ac, duration=1346.261s, table=73, n_packets=0, n_bytes=
23 cookie=0xc51dc48d, duration=1346.266s, table=73, n_packets=0, n_bytes=
24 cookie=0x956323ac, duration=1346.264s, table=73, n_packets=12, n_bytes
25 [root@node-4 bpf]#
26 [root@node-4 bpf]# ovs-ofctl dump-flows br-int table=74
27 cookie=0xc51dc48d, duration=1348.770s, table=74, n_packets=76, n_bytes
28 cookie=0xc51dc48d, duration=1348.769s, table=74, n_packets=0, n_bytes=
29 cookie=0x956323ac, duration=1348.767s, table=74, n_packets=0, n_bytes=
30 cookie=0x956323ac, duration=1348.756s, table=74, n_packets=0, n_bytes=
31
32 cookie=0xc51dc48d, duration=1348.769s, table=74, n_packets=36, n_bytes
33 cookie=0xc51dc48d, duration=1348.756s, table=74, n_packets=0, n_bytes=
34
35 cookie=0x956323ac, duration=1348.769s, table=74, n_packets=0, n_bytes=
36 cookie=0x956323ac, duration=1348.766s, table=74, n_packets=0, n_bytes=
37 cookie=0x956323ac, duration=1348.763s, table=74, n_packets=0, n_bytes=
38 cookie=0xc51dc48d, duration=1348.759s, table=74, n_packets=0, n_bytes=
39 cookie=0xc51dc48d, duration=1348.759s, table=74, n_packets=33, n_bytes
40 cookie=0x956323ac, duration=1348.768s, table=74, n_packets=0, n_bytes=
41 cookie=0xc51dc48d, duration=1348.766s, table=74, n_packets=0, n_bytes=
42 cookie=0xc51dc48d, duration=1348.762s, table=74, n_packets=0, n_bytes=

```

```

43 cookie=0x956323ac, duration=1348.754s, table=74, n_packets=0, n_bytes=
44 cookie=0x956323ac, duration=1348.766s, table=74, n_packets=0, n_bytes=
45 cookie=0xc51dc48d, duration=1348.757s, table=74, n_packets=0, n_bytes=

```

通过查看table 74的openflow流表，结合classifier的查找，最终设置的nd_target和nd_ttl是通过table 74的以下openflow流表：

```

1 cookie=0xc51dc48d, duration=1348.769s, table=74, n_packets=36, n_bytes=
2 cookie=0xc51dc48d, duration=1348.756s, table=74, n_packets=0, n_bytes=0

```

需要分析classifier的查找过程，具体查看该文档：[ovs的classifier创建和查询](#)

ovs相关patch

按照之前classifier原理，如果后一个stage map没有匹配上，不会将后一个stage map的字段设置到wc中的。比如table 74本来是不匹配我们的packet的，所以不应该设置上 `nd_target` 和 `nd_ttl`。但是这里却被设置上了，其原因是社区合入了这个patch：[classifier: Fix missing masks on a final stage with ports trie. · openvswitch/ovs@489553b](#)

目前已经向社区提问：[the dumped WC and generated WC mismatch for ipv6 · Issue #375 · openvswitch/ovs-issues](#)

此外，这里经过classifier后，虽然产生了 `nd_target` 和 `nd_ttl`，但是最终下发到datapath flow是没有 `nd_target` 和 `nd_ttl` 的，因为我们这里的icmp_type=128,icmp_code=0。因此revalidator sweep时，会将dp flow dump上来，然后进行一下xlate到classifier，从而生成了xlate mask，同时取出datapath mask，通过比较datapath mask和xlate mask时不一致，从而会删除dp flow。

datapath flow和生成的megaflow的mask不一致，为什么revalidator没有持续的快速删除呢？

我们通过bpfttrace定位观察到，当datapath flow和生成的megaflow的mask不一致，revalidator没有持续的快速删除。其原因是revalidator没有触发到执行 `revalidate_ukey__` 函数的条件。

```

1
2 /* Verifies that the datapath actions of 'ukey' are still correct, and
3  * 'stats' for it.
4  *
5  * Returns a recommended action for 'ukey', options include:
6  *     UKEY_DELETE The ukey should be deleted.
7  *     UKEY_KEEP   The ukey is fine as is.
8  *     UKEY_MODIFY The ukey's actions should be changed but is otherwise
9  *                 fine. Callers should change the actions to those found
10 *                 in the caller supplied 'odp_actions' buffer. The
11 *                 recirculation references can be found in 'recircs'
12 *                 must be handled by the caller.
13 *
14 * If the result is UKEY_MODIFY, then references to all recirc_ids used
15 * new flow will be held within 'recircs' (which may be none).
16 *
17 * The caller is responsible for both initializing 'recircs' prior this
18 * and ensuring any references are eventually freed.
19 */
20 static enum reval_result
21 revalidate_ukey(struct udpif *udpif, struct udpif_key *ukey,
22                const struct dpif_flow_stats *stats,
23                struct ofpbuf *odp_actions, uint64_t reval_seq,
24                struct recirc_refs *recircs, enum flow_del_reason *del_reason)
25 {
26     OVS_REQUIRES(ukey->mutex)
27     {
28         bool need_revalidate = ukey->reval_seq != reval_seq;
29         enum reval_result result = UKEY_DELETE;
30         struct dpif_flow_stats push;
31
32         ofpbuf_clear(odp_actions);
33
34         push.used = stats->used;

```



```

34     push.tcp_flags = stats->tcp_flags;
35     push.n_packets = stats->n_packets - ukey->stats.n_packets;
36     push.n_bytes = stats->n_bytes - ukey->stats.n_bytes;
37
38     if (stats->n_packets < ukey->stats.n_packets &&
39         ukey->stats.n_packets < UINT64_THREE_QUARTERS) {
40         /* Report cases where the packet counter is lower than the prev
41          * instance, but exclude the potential wrapping of an uint64_t.
42         COVERAGE_INC(ukey_invalid_stat_reset);
43         log_unexpected_stats_jump(ukey, stats);
44     }
45
46     if (need_revalidate) { // 这里需要ukey->reval_seq != reval_seq, 即ukey
47         if (should_revalidate(udpif, ukey, push.n_packets)) {
48             if (!ukey->xcache) {
49                 ukey->xcache = xlate_cache_new();
50             } else {
51                 xlate_cache_clear(ukey->xcache);
52             }
53             result = revalidate_ukey__(udpif, ukey, push.tcp_flags,
54                                     odp_actions, recircs, ukey->xcac
55                                     del_reason); // 因为revalidate_uk
56         } else {
57             /* Delete, since it is too expensive to revalidate. */
58             *del_reason = FDR_T00_EXPENSIVE;
59         }
60     } else if (!push.n_packets || ukey->xcache
61               || !populate_xcache(udpif, ukey, push.tcp_flags)) {
62         result = UKEY_KEEP;
63     }
64
65     /* Stats for deleted flows will be attributed upon flow deletion. S
66     if (result != UKEY_DELETE) {
67         xlate_push_stats(ukey->xcache, &push, ukey->offloaded);
68         ukey->stats = *stats;
69         ukey->reval_seq = reval_seq;
70     }
71
72     return result;
73 }

```

分析到我们这里的例子，其实都是一个正常的流量，没有涉及到超过max_idle等情况的删除操作。所以这里能否执行 `revalidate_ukey__` 函数，需要看 `need_revalidate` 变量和 `should_revalidate` 函数的判断。这里的 `should_revalidate` 函数返回的是true。我们主要关注 `need_revalidate` 变量。

`need_revalidate` 变量的结果是通过比较ukey和udpif的reval_seq的值。一般情况下，他们是相等的。除非udpif的reval_seq的值发生变化，即执行 `udpif_revalidate` 函数调用。只有 `backer->need_revalidate` 有值才会执行 `udpif_revalidate` 函数。

而 `backer->need_revalidate` 会被设置以下的情况：

```

1  /* Reasons that we might need to revalidate every datapath flow, and
2   * corresponding coverage counters.
3   *
4   * A value of 0 means that there is no need to revalidate.
5   *
6   * It would be nice to have some cleaner way to integrate with coverage
7   * counters, but with only a few reasons I guess this is good enough for
8   * now. */
9  enum revalidate_reason {
10     REV_RECONFIGURE = 1,          /* Switch configuration changed. */
11     REV_STP,                     /* Spanning tree protocol port status ch
12     REV_RSTP,                    /* RSTP port status change. */
13     REV_BOND,                    /* Bonding changed. */
14     REV_PORT_TOGGLED,            /* Port enabled or disabled by CFM, LACP
15     REV_FLOW_TABLE,              /* Flow table changed. */

```

```
16     REV_MAC_LEARNING,          /* Mac learning changed. */
17     REV_MCAST_SNOOPING,        /* Multicast snooping changed. */
18 };
```

对于我们的环境，我们观察到主要有 `REV_FLOW_TABLE` 和 `REV_MAC_LEARNING` 的变化引起 `udpif_revalidate` 函数调用，从而引起 `ukey` 和 `udpif` 的 `reval_seq` 不一致，最终 `need_revalidate` 变量为 `true`，从而进行 `revalidate_ukey__` 函数调用。最终的结果是 `result` 变量为 `UKEY_DELETE`，`del_reason` 为 `FDR_FLOW_WILDCARDED`，最终会删除 `dp flow`。